

A digital geometry for hexagonal pixels

Sarah B M Bell, Fred C Holroyd* and David C Mason

Comparison of hexagonal and square pixels and arrays for image processing shows that the former have many advantages. However, squares can be addressed with integers and orthogonal axes, while for hexagons the axes must be at an oblique angle of 60°. This paper describes a general method of producing geometrical algorithms for such axes.

Keywords: hexagonal arrays, hexagonal tessellations, digital geometry

Square pixels and arrays as shown in Figure 1a (abbreviated here as simply 'squares') are commonly used in the related fields of image processing, geographical information systems and signal detection. This may be due to the historical accident that devices such as detectors and displays were originally built for squares before other possibilities had been considered. Subsequently compatibility with existing hardware may have ensured their continued use.

This paper considers whether there are any other valid reasons for choosing squares. Hexagonal pixels and arrays as shown in Figure 1b (abbreviated here as 'hexagons') are the other obvious choice. For many of the approaches explored in this paper, it is the position of the centroid of the pixels which is important and not the shape of the pixel. The square or the hexagonal pixel is in one sense the natural representation for a square or hexagonal grid of centroids since it is the Dirichlet tessellation. Nevertheless, for example, pixels whose centroids are on a hexagonal grid may be represented as rectangles, Figure 1c or 60/120° rhomboids, as in Figure 1d, and it makes no difference to the geometry which will be presented.

If however pairs of pixels from a square grid are amalgamated, to obtain a grid of 2×1 rectangles, and the grid of centroids of these rectangles is taken, as in Figure 1e, then this paper and its

references will not be applicable because the grid is not hexagonal. The only way of turning a square grid into an hexagonal grid is to resample the data with some suitable interpolation algorithm.

There are advantages in capturing the data in hexagonal form, as appears below in the survey of existing literature. Having pointed out that it is the centroids rather than the edges of the pixels that count most, the literature which demonstrates that hexagons have many advantages is examined. The paper then considers in detail a method of removing a remaining disadvantage of hexagons.

Hexagonal pixels are uniformly adjacent to their neighbours along the six sides of the pixel, as shown in Figure 2a or, equivalently, the centroid of the middle pixel is the same distance from the centroids of the six adjacent pixels. This makes image processing algorithms, such as finding connected components in a raster image, easier¹. Square pixels are not uniformly adjacent. As shown in Figures 2b and 2c, squares are adjacent to four neighbours along a side and four neighbours at the corners or, equivalently, the centroid of the middle pixel is at different distances from the neighbours at 90° and 45°. Golay¹ also notes that hexagons offer greater angular resolution as the nearest neighbours of the same type are separated by 60° instead of 90°. Preston² shows that Golay's image processing algorithms are useful in practice. Bell *et al.*³ point out that the average distance from a point in the pixel to the centroid for a pixel of unit area is approximately 0.377 for hexagons but approximately 0.388 for squares. Thus the centroid of a square is a worse approximation to the whole pixel than the centroid of a hexagon. Digital topology has been developed by Rosenfeld^{4,5} who shows that squares necessarily lead to the formation of algorithms which must consider both the neighbours along the four sides of the square and those at its four corners, and treat them as separate cases. Deutsch⁶ demonstrates that this difficulty does not occur for hexagons. This point may also be made within the language of classical algebraic topology^{7,8} and an example is interpolated at the end of this review of the literature. Deutsch⁹ also concludes that the complexity of thinning algorithms for raster data decreases as the types of adjacency become fewer. After consideration of many other

NERC Unit for Thematic Information Systems, Geography Department, University of Reading, Whiteknights, PO Box 227, Reading RG6 2AB, UK

*Faculty of Mathematics, The Open University, Walton Hall, Milton Keynes MK7 6AA, UK

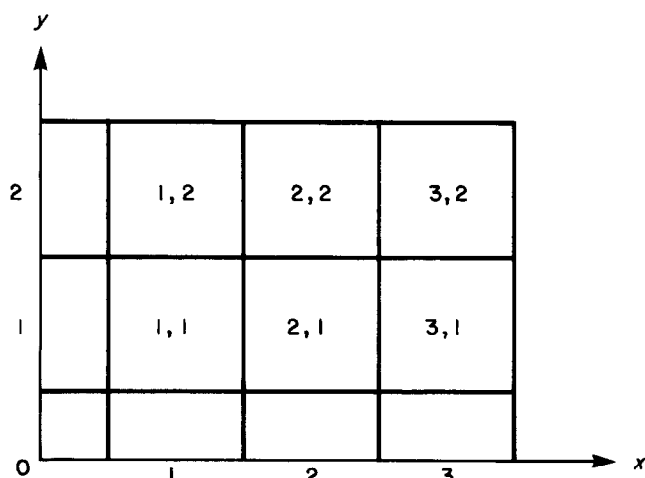


Figure 1a. Square pixels in a square array

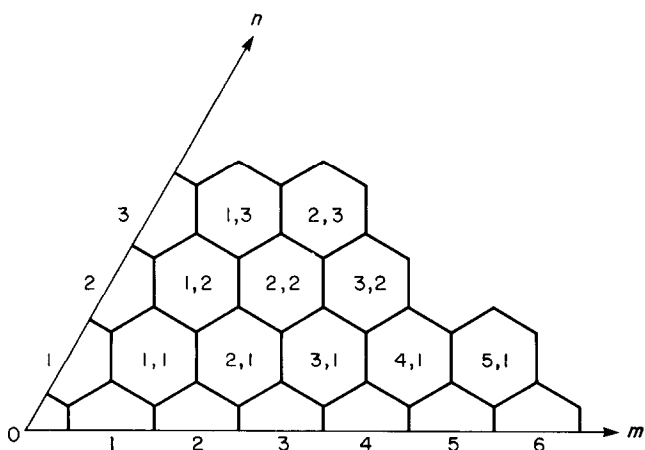


Figure 1b. Hexagonal pixels in a hexagonal array

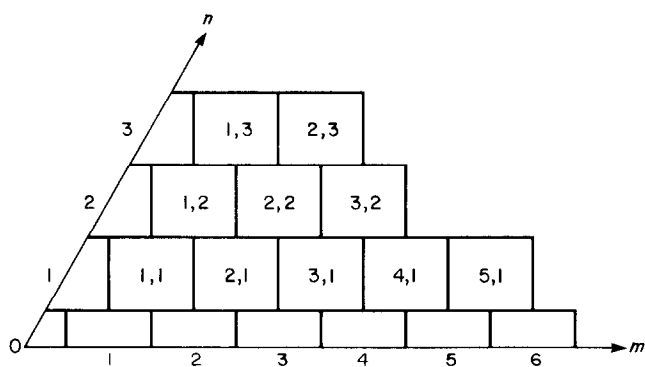


Figure 1c. Rectangular pixels in a hexagonal array

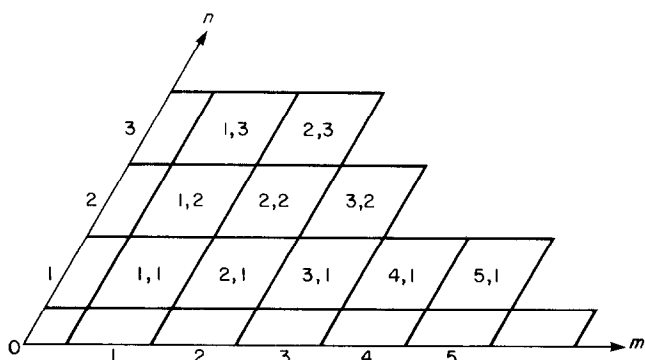


Figure 1d. Rhomboidal pixels in a hexagonal array

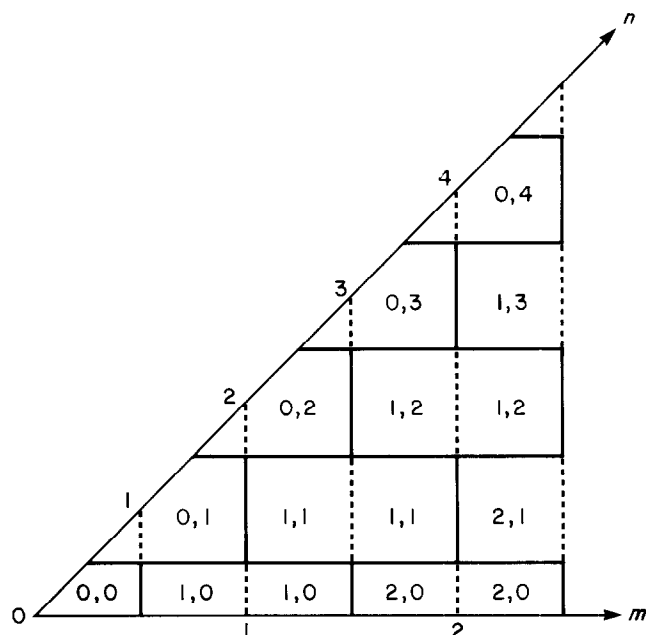


Figure 1e. Square pixels forming a rectangular array

factors, Deutsch concludes that hexagons are optimal for thinning. Luczak and Rosenfeld¹⁰ suggest the use of oblique axes for addressing hexagons and show that it is possible to define a metric for such axes known as the 'hexagonal distance'. Their type of addressing is the same as that adopted in this paper and therefore their result is of direct use. Mersereau¹¹ discusses image detectors and shows that hexagonal sampling is the optimal sampling scheme for signals which are band limited over a circular region of the Fourier plane. For this reason, and because the hexagon has more axes of symmetry than the square, algorithms for processing the signal are more efficient using hexagons. The discrete Fourier transform is one such algorithm. Cox¹² shows that the advantages of hexagonal detector arrays extend to point-source tracking. Hexagons can also be used for hierarchical image processing^{3,13,14} in the same way as the quadtree¹⁵ for squares.

The advantages of hexagons, explored under the heading of digital topology⁴⁻⁶, can also be framed in the classical language of algebraic topology. Homotopy^{7,8} is preserved in some useful transformations of the hexagonal grid while it is not on the square grid. In each of Figures 3a, 3b, 3c and 3d a topology is defined on the set composed of the points within $ABCD$ by making set S the closed set consisting of black pixels including boundaries. Figure 3a represents the initial condition for squares, Figure 3b for hexagons. Clearly the closed loops L_1 and L_2 are homotopic^{7,8}, i.e. L_1 can be gradually and continuously transformed into L_2 within the set S . The boundary is now thinned^{1,2,4,5,9} in each case, producing Figures 3c and 3d. In Figure 3c the loops L_1 and L_2 are no longer homotopic, although S remains connected, while in Figure 3d they remain homotopic. Thus homotopy has not been preserved for the square pixels over the thinning transformation. It is worth remarking that the fundamental difference between the topologies of the hexagonal-pixel and square-pixel arrangements is that the hexagonal-pixel topology is actually the topology of the plane, whereas the square-pixel topology is not. Either one must count

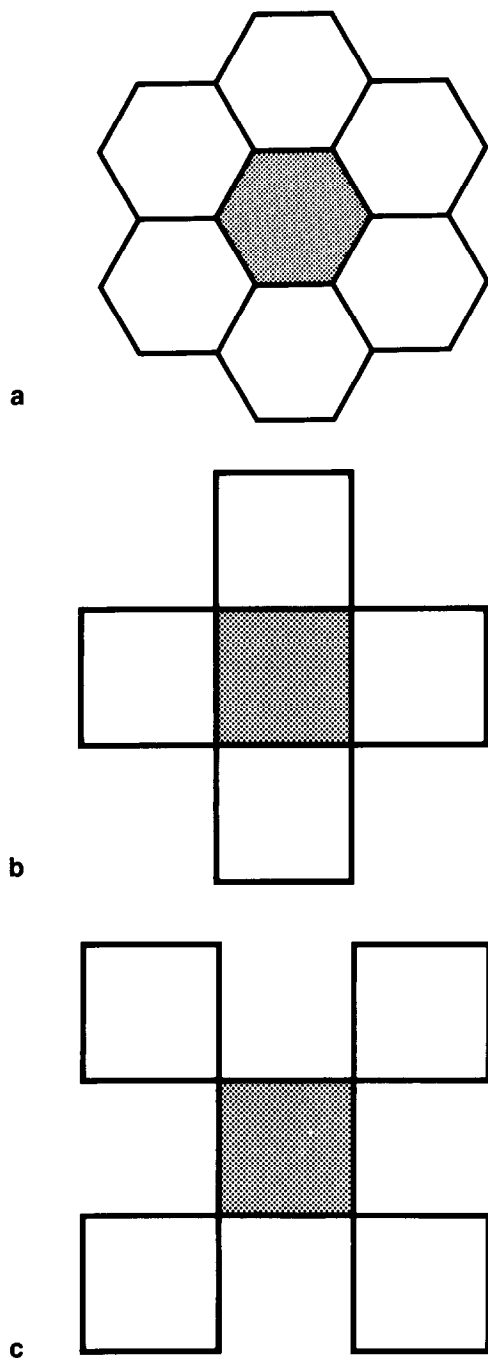


Figure 2. **a**, Neighbours of a hexagonal pixel or array; **b**, nearest neighbours of a square pixel or array; **c**, next-nearest neighbours of a square pixel or array

diagonally adjacent pixels as topologically connected or one must not. In the former case, there must be a topological 'bridge' at each vertex to ensure that both pairs of opposite pixels are connected (producing a topology that certainly does not obey Jordan's theorem); in the latter case, there must be a 'hole' at each vertex so that although Jordan's theorem *does* apply here, there are infinitely many homotopically non-equivalent loops.

Squares still have one advantage. Square pixels can be more easily addressed with integers referred to two orthogonal axes (Figure 1 shows only the positive quadrant in each case but the whole plane may be addressed by using negative coordinates). The address

is written (x, y) . The usual version of Cartesian geometry (Cartesian is taken as implying the orthogonality of the axes) can then be employed for spatial calculations. In this x and y appear separately and explicitly. For example the equation of a circle centred on the origin of radius r is given by:

$$x = (r^2 - y^2)^{1/2}$$

and a linear transform is given by the matrix equation:

$$\begin{vmatrix} x' \\ y' \end{vmatrix} = \begin{vmatrix} a & b \\ c & d \end{vmatrix} \begin{vmatrix} x \\ y \end{vmatrix}$$

The result of such a geometrical calculation yields a pixel address by simple rounding or truncation to integers.

If hexagonal pixels are referred to such orthogonal axes the address of the pixels cannot contain more than one integer and the second number is irrational. For example, a typical address might be $(1, 3^{1/2})$. However, the hexagonal pixels can be addressed by integers if the axes are oblique, enclosing an angle of 60° . (See Figure 1b.) The problem then is to find an easy and elegant way of performing geometrical calculations such as those of the last paragraph. It would be possible to convert an integer address into an address referred to orthogonal axes, perform the calculation, and then to convert the result back to oblique axes. This requires floating-point, complicates any error analysis, is not elegant and is less efficient. Instead, the addresses of the pixels (m, n) , m, n integer and referred to oblique axes, can be mapped onto complex numbers of the form $m + n/2 + n * i * 3^{1/2}/2$ to provide a method of performing calculations involving m and n which can be performed using the conventional arithmetic instructions of the conventional computer. This set of algorithms is called set A and they are found in the following section headed 'Mapping the pixel addresses to complex numbers'. The geometrical operations in their turn are expressed as combinations of primitive operations on complex numbers, and this set of algorithms is called set B. They are found in the sections headed 'Creating algorithms for the geometry' to 'General linear transform' inclusive. Thus the combined set of algorithms, A and B, work directly on pixel addresses (m, n) and yield other such pixel addresses by simple rounding or truncation to integers. It is equally easy to solve geometrical problems within the new approach and this is demonstrated by several examples.

IMPLEMENTATION

The method of holding the hexagonal pixel addresses (m, n) referred to the oblique axes is first considered. Clearly they can be held in an integer array, pixel, declared in a high-level language of the host machine, and dimensioned: pixel[m'][n'] (in the notation of the computer language C, where m' and n' are the maximum values of m and n in the image).

The algorithms of set A and B can be implemented in two ways, and the two methods can be mixed in the same program. First, method (i), the algorithms of set

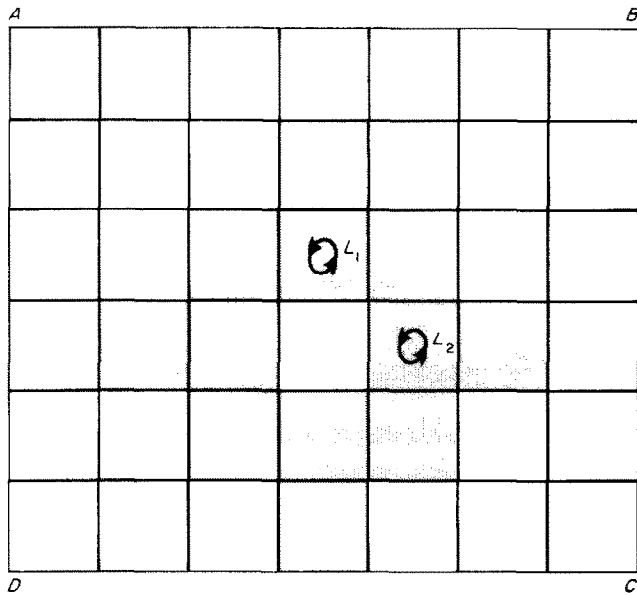


Figure 3a. Square pixels: L_1, L_2 homotopic

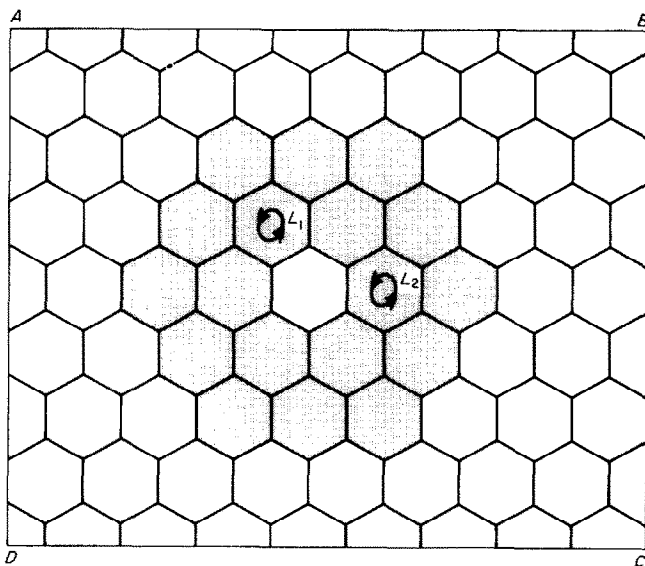


Figure 3b. Hexagonal pixels: L_1, L_2 homotopic

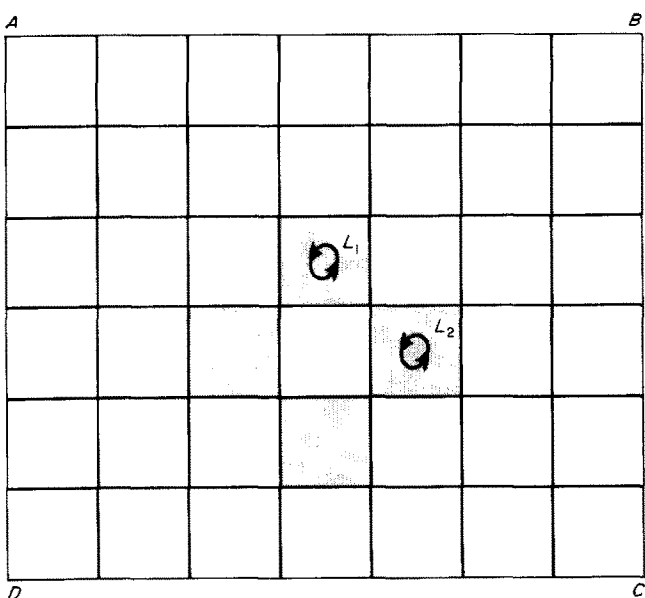


Figure 3c. Thinned square pixels: L_1, L_2 not homotopic

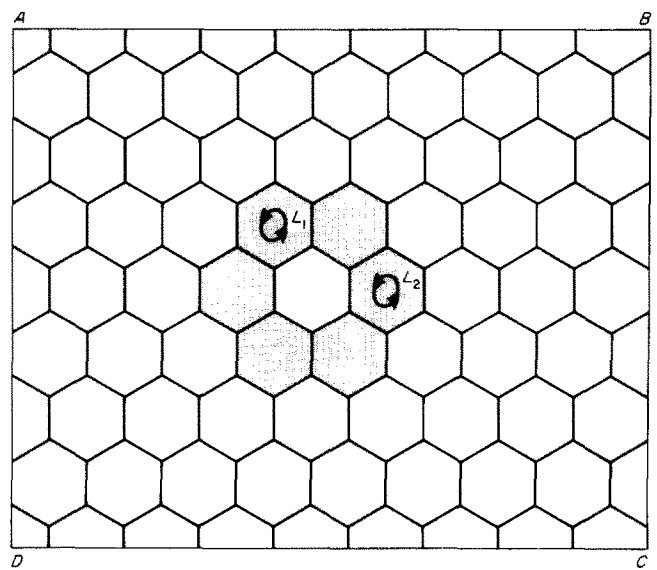


Figure 3d. Thinned hexagonal pixels: L_1, L_2 homotopic

A, as displayed in the next section, use only conventional integer or real operations on pixel addresses (m, n) defined in the oblique axes. They can thus be implemented as subroutines within the host high-level language. (There is no requirement that the host language should support operations on complex numbers.) The algorithms of set B are then implemented by changing each complex-number operation: $+$, $-$, $*$, $/$, $\sim$, and extraction of a square root, displayed in the formula into a call to the relevant subroutine of set A.

The second method of implementing set A and set B, method (ii), is to consider A and B as formulae and replace each variable in a set B by a suitable pixel address (m, n) . The complex-number operations $(+, -, *, /, \sim)$, and extraction of a square root) on the variables are then replaced by the formulae of set A and the result simplified as far as possible. At this point the operations $(+, -, *, /, \text{and extraction of a square root})$ are again those of the integers and reals, and the simplified formulae for the unknown pixel addresses can be replaced by code in the host high-level language.

The first method makes programming easier but the second makes execution faster. This point is considered again in the section headed 'Efficiency considerations' where a worked example of both methods of implementation is provided.

MAPPING PIXEL ADDRESSES TO COMPLEX NUMBERS

The formulae of set A is found in this section. The pixels are addressed by integers referred to axes at 60° as shown in Figure 1b. Suppose the pixel address is $P = (m, n)$, m, n integer. This is mapped onto the two complex numbers a and b such that:

$$m \rightarrow m * a$$

$$n \rightarrow n * b$$

where

$$a = 1$$

$$b = 1/2 + i * 3^{1/2}/2$$

so

$$P = m * a + n * b$$

Vector addition of pixel addresses maps onto complex-number addition.

$$(m * a + n * b) + (m' * a + n' * b) = M_a * a + N_a * b$$

where

$$M_a = m + m', \quad N_a = n + n' \quad (1)$$

It can be seen that if the operands are integers then so will be the answer (M_a, N_a). A similar property holds for vector subtraction.

$$(M * a + n * b) - (m' * a + n' * b) = M_s * a + N_s * b$$

$$M_s = m - m', \quad N_s = n - n' \quad (2)$$

where M_s and N_s are integers.

More surprisingly, this property holds true of complex multiplication, that is, the pixel addresses form a ring under complex multiplication. This can be approached by considering the products of a and b :

$$a * a = a, a * b = b * a = b, b * b = -a + b \quad (3)$$

Thus:

$$(m * a + n * b) * (m' * a + n' * b) = M_m * a + N_m * b$$

$$M_m = m * m' - n * n' \quad (4)$$

$$N_m = (m + n) * n' + m' * n \quad (4)$$

where M_m and N_m are integers. The geometrical effect of a complex number multiplication by (m, n) is to rotate the vector (m', n') by an angle, z' whose tangent is $(m + 2 * n) / (3^{1/2} * m)$ and to scale it by $3^{1/2} * (m * m + n * m + n * n)^{1/2}$.

The complex conjugate of a complex number $u + i * v$ is defined by:

$$(u + i * v)^{\sim} = u - i * v$$

where the symbol $\sim$ is used to denote complex conjugation throughout this paper. It is assumed that the reader is familiar with the algebraic properties of complex conjugation. Geometrically, taking the complex conjugate of a number reflects it in the line defined by the vector from the origin to a : $(1, 0)$. This means that the complex conjugate of a number ($C^{\sim}$) scales by the same amount as the number (C) when it multiplies another (B), and $C^{\sim}$ rotates B by an angle of the same magnitude as C but in the opposite direction. The complex conjugate of a pixel address is also another integer pixel address. We have $a^{\sim} = a$ and $b^{\sim} = a - b$. Thus:

$$(m * a + n * b)^{\sim} = M_c * a + N_c * b$$

$$M_c = m + n, \quad N_c = -n \quad (5)$$

Multiplying any complex number z by its complex conjugate yields a number whose imaginary part is zero and whose real part is the square of the distance of z from the origin. Thus using equations (3), (4) and (5), it is found that the distance from the origin of the pixel with address (m, n) is

$$d^2 = m * (m + n) + n^2 \quad (6)$$

The division of one pixel address by another (geometrically a rotate and scale) does not provide an integer answer. However, the division can be reduced to that of the division of one integer by another, and so the integer capabilities of the computer may be used if desired. We have, for $U = m * a + n * b$ and $V = m' * a + n' * b$:

$$U/V = U * V^{\sim} / (V * V^{\sim}) = M_d * a + N_d * b$$

$$M_d = \{m * (m' + n') + n * n'\} / \{m' * (m' + n') + n'^2\} \quad (7)$$

$$N_d = \{n * m' - m * n'\} / \{m' * (m' + n') + n'^2\} \quad (7)$$

the numerators and denominators being integer. The complex numbers (including all the numbers $(m * a + n * b)$ where m and n are not integers) constitute a field, so that $1/U$, U complex, is uniquely defined and always exists unless U is zero.

The square root of a pixel address is considered. If (m, n) is the square of (N_r, M_r) by substitution in equation (4) we have:

$$N_r^2 - M_r^2 = n \quad (8)$$

$$2 * N_r * M_r + M_r^2 = m \quad (9)$$

Solving the simultaneous equations (8) and (9) we obtain:

$$M_r = \pm [\{(m - n) + 2 * (m^2 + m * n + n^2)^{1/2}\} / 3]^{1/2} \quad (10)$$

$$N_r = \pm [\{-(n + 2 * m) + 2 * (m^2 + m * n + n^2)^{1/2}\} / 3]^{1/2} \quad (10)$$

The method above will always work whether or not the pixel address is negative in m or n or both. However, as shall be seen in the examples of set B below, sometimes it is useful to know how to replace i , (M_i, N_i) , the square root of minus one, by a pixel address.

Choosing the positive square root in equations (10), the following is obtained:

$$M_i = -1/3^{1/2}, \quad N_i = 2/3^{1/2} \quad (11)$$

The algorithms defined by equations (1), (2), (4), (5), (6), (7), (10), (11) are called set A.

CREATING ALGORITHMS FOR GEOMETRY

Let us now move on to finding the algorithms of set B. This is done using the known properties of the complex numbers. For example, if one wants to translate an

image by a vector A that maps a pixel of address P to a pixel of address Q one can find the right vector A by considering A , P and Q to be complex numbers and solving the following complex-number equation:

$$P + A = Q$$

giving:

$$A = Q - P$$

Let the pixel addresses of P , Q , A be (m', n') , (m, n) , (M_s, N_s) . In the event that set A and B are implemented by method (i) in the section on 'Implementation' this can be translated into a subroutine call:

$$M_s = \text{fsubm}(m, n, m', n')$$

$$N_s = \text{fsubn}(m, n, m', n')$$

fsubm and fsubn apply the correct formula, equation (2) in set A, to produce the result of a subtraction. If set A and set B are implemented by the second method, direct substitution of equation (2) gives:

$$A = (m' - m, n' - n)$$

where the minus signs are now operating on ordinary integers and not on complex numbers. Every pixel in the image, X , may now be transformed according to the complex number equation:

$$X' = X + A \quad (12)$$

As can be seen, complex numbers only enter into the method from the point of view of calculating the right algorithm to produce the change in the image required. Once the algorithm of set B can be found in terms of complex-number primitive operations, the algorithms of set A define the arithmetic to be carried out on the pixel addresses.

As another example, suppose one wishes to rotate and scale an image so that pixel P falls on pixel Q . This is the geometric operation which maps to complex-number multiplication. Suppose the complex-number (or alternatively vector) required is B , the following equation has to be solved:

$$P * B = Q$$

This may be done with complex-number division:

$$B = Q/P$$

Once again, one may find B by calling the relevant subroutine based on equation (7) in set A, or by explicit algebraic substitution from equation (7). Then each pixel of the image, X , must be transformed to a new pixel, X' , according to the formula:

$$X' = X * B \quad (13)$$

A further selection of algorithms is now provided for an assortment of geometrical problems, which are referred to as a set B. Only the complex-number geometry will be given. The algorithms can be realised by either method (i) or (ii) in the section on 'Implementation'.

DRAWING REGULAR POLYGONS

Let us start with a simple example illustrated in Figure 4. The pixel address of point C needs to be found given the pixel addresses of points A and B , and that ABC is an equilateral triangle. Suppose multiplying by an unknown complex-number X will rotate the line CA

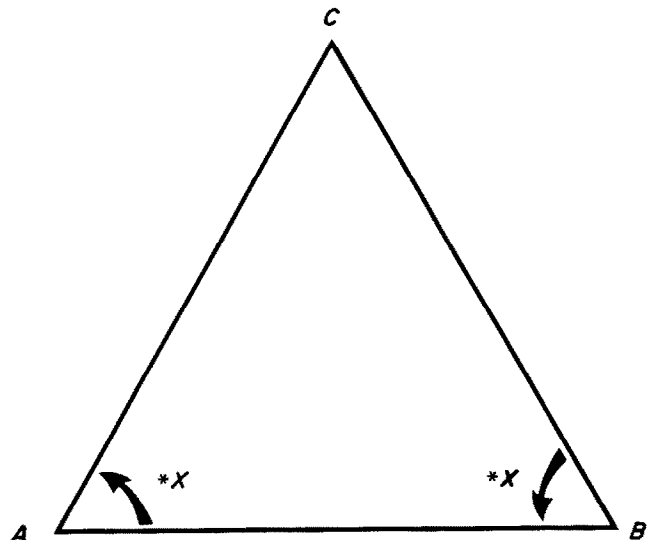


Figure 4. Multiplication by X rotates BA to CA and CB to AB

about A until it falls on BA . This can be converted to the complex-number equation:

$$B - A = X * (C - A) \quad (14)$$

However, exactly the same geometric operation will rotate the line AB about B until it falls on the line CB . The following may therefore also be written:

$$C - B = X * (A - B) \quad (15)$$

Equations (14) and (15) are two simultaneous equations from which X may be eliminated to obtain:

$$C = \{A + B \pm (-3)^{1/2} * (A - B)\} / 2 \quad (16)$$

The solution for drawing an equilateral triangle might be intuitively obvious if it is known that such a triangle has internal angles of 60° , and is therefore a special case and particularly simple to construct if the axes are also at 60° . However, the method used in the last section can be extended to other regular polygons. The complex algebra does not depend on the type of grid used to create the pixels. It is only the meaning given to the complex-number primitives in set A which depends on the type of grid. The conditions for extending the approach described in this paper to non-hexagonal grids are given in Bell *et al.*¹⁶. The previous method is now demonstrated with a square.

Given A and B it is required to find C and D such that $ABCD$ is a square. Suppose multiplying by an unknown complex-number X will rotate the line DA about A until it falls on BA , AB about B giving CB , and BC about C giving DC . This can be converted into the

complex-number equations:

$$(D - A) * X = B - A \quad (17)$$

$$(A - B) * X = C - B \quad (18)$$

$$(B - C) * X = D - C \quad (19)$$

Equations (17) to (19) are considered as simultaneous equations from which first X and then D need to be eliminated. We obtain:

$$C = B \pm \{-(A - B)^2\}^{1/2} \quad (20)$$

By substituting D for C , C for B , B for A in equation (20) D may then be obtained.

Equation (20) could also have been arrived at directly, by noting that multiplication of a pixel address by i or $-i$ rotates the line from the origin to the pixel address by 90° about the origin in an anticlockwise or clockwise direction. The regular j -gon can be solved with less effort as follows. Suppose we are given one side of the j -gon, AB , and the number of sides, j . Let the centre of the j -gon be O , and the multiplier which converts OA into OB be X . Then:

$$X * (A - O) = (B - O) \quad (21)$$

and

$$X^j * (A - O) = A - O$$

so X is a j -th root of unity. Therefore:

$$X = \cos(2 * \pi / j) \pm i * \sin(2 * \pi / j)$$

Equation (21) can now be solved to find O , and the other vertices of the j -gon are easily generated by multiplying BO by X to an integer power between 1 and $j - 2$.

DRAWING AN ISOSCELES TRIANGLE

Given A and B it is required to find C such that the sides AC and BC will be of equal length d . We have:

$$(C - A) * (C - A)^{\sim} = d^2 \quad (22)$$

$$(C - B) * (C - B)^{\sim} = d^2 \quad (23)$$

Equations (22) and (23) are considered as a pair of simultaneous equations from which $C^{\sim}$ needs to be eliminated. We obtain:

$$C = (A + B)/2 \pm \{[(A - B)^{\sim} * (A - B) * [(A - B)^{\sim} * (A - B) - 4 * d^2]\}^{1/2} / [2 * (A - B)^{\sim}]\}$$

We may go further and use equation (6) of set A to parametrize the length of AB as d' . We obtain:

$$C = (A + B)/2 \pm [(A - B) * \{1 - 4 * d^2 / d'^2\}]^{1/2} / 2 \quad (24)$$

It can be clearly seen that equation (24) reduces to equation (16) when $d' = d$.

EQUATION OF A CIRCLE

This can be deduced from equation (24). C is the known centre of the circle, and A a known point on the circumference of the circle. It can be seen that the radius d of the circle is given by:

$$d^2 = (A - C)^{\sim} * (A - C)$$

Every point B on the circumference will form an isosceles triangle with A and C , of equal sides AC and CB . But this is exactly the triangle that is described in equation (24), where d' is now the length of the varying chord AB . Equation (24) is manipulated to find B in terms of C and A :

$$B = [2 * C - A * \{1 \pm \{1 - 4 * d^2 / d'^2\}^{1/2}\}] / [1 \pm \{1 - 4 * d^2 / d'^2\}^{1/2}] \quad (25)$$

The circle is now created by varying (d/d') between $1/2$ and infinity.

Alternatively, the ratio d/d' can be fixed at some conveniently large value, p , and the circle approximated with a set of short chords. This can be done using equation (20) and its derivatives, however, we would like to avoid calculating trigonometric functions for every point on the circle. The following alternative algorithm based on equation (25) is therefore proposed:

Initialize:

$$h' = \{1 - 4 * p * p\}^{1/2}$$

$$h = 1 - h'$$

$$g = (1 + h')/h$$

$$g' = 2 * C/h$$

output A

Loop until {absolute value of $(B - A)$ } is less than 1

output B

$$A = B$$

$$B = g' - A * g$$

End loop.

The error in calculating the vertices of the chords by this method will be equal to the number of iterations through the loop, p , times the error in a single iteration, e , so $e * p$ must be less than 0.5 if the result is to be correct to the nearest pixel.

EQUATION OF A STRAIGHT LINE

Let us now find the equation of a straight line that passes through two known points, A and C . Let X be any point on this straight line. Then it is seen that $(X - C)$ must be some real number times $(A - C)$. Let this number be x . Then:

$$X = x * (A - C) + C \quad (26)$$

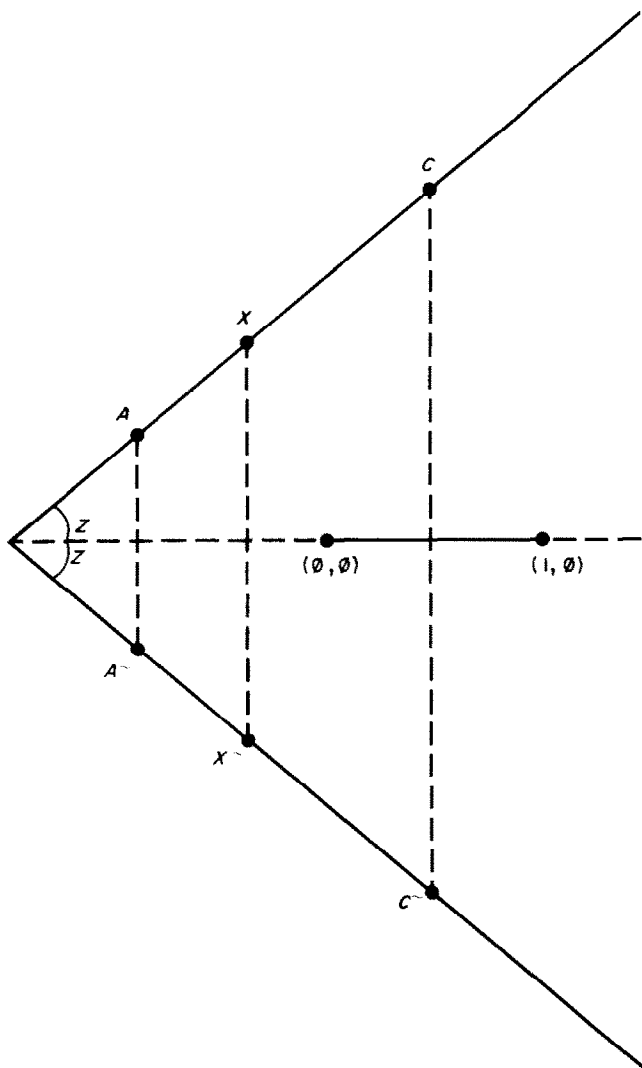


Figure 5. Conjugation of $X = x * (A - C) + C$ gives $X^{\sim} = x * (A^{\sim} - C^{\sim}) + C^{\sim}$

With the A and C shown in Figure 5, values of x less than zero yield points on the line above C , values of x between zero and one yield points on the line between C and A , while values of x greater than one yield points below A . As Figure 5 shows, the reflection of our line can easily be found in the line $(0, 0)$ to $(1, 0)$ by taking complex conjugates:

$$X^{\sim} = x * (A - C)^{\sim} + C^{\sim} \quad (27)$$

Equations (26) and (27) may now be regarded as two simultaneous equations and x be eliminated. This will be useful in the next section. We obtain:

$$(X - C) * (A - C)^{\sim} = (A - C) * (X - C)^{\sim} \quad (28)$$

INTERSECTION OF TWO STRAIGHT LINES

The first straight line is taken to be that discussed in the last section. The second straight line is assumed to pass through two known points B and D . Let the point of intersection of the two lines be X . An equation analogous to equation (28) may then be written for the

second straight line:

$$(X - D) * (B - D)^{\sim} = (B - D) * (X - D)^{\sim} \quad (29)$$

Equations (28) and (29) are now considered as two simultaneous equations and $X^{\sim}$ is eliminated, obtaining:

$$X = [(A - C) * (D^{\sim} * B - D * B^{\sim}) - (B - D) * (C^{\sim} * A - C * A^{\sim})] / [(A - C)^{\sim} * (B - D) - (B - D)^{\sim} * (A - C)] \quad (30)$$

TRANSLATION, ROTATION AND SCALING

It is supposed that each pixel in the image is to be translated, rotated and scaled by the same amount that maps the pixel addressed by R onto the pixel addressed by S , and the pixel addressed by J onto the pixel addressed by K . This is an extension to the two operations described in the section on 'Creating algorithms for geometry', and, as such, must be represented by some translation, U , and rotation and scaling V . We have:

$$S = V * R + U$$

$$K = V * J + U$$

These are regarded as two simultaneous equations in the unknowns U and V , and solved to get:

$$V = (S - K) / (R - J) \quad (31)$$

$$U = (K * R - S * J) / (R - J) \quad (32)$$

Then each pixel, address X , in the image must be transformed as follows:

$$X' = X * V + U$$

LINEAR TRANSFORM

The linear transform in matrix form is given by:

$$\begin{vmatrix} x' \\ y' \end{vmatrix} = \begin{vmatrix} r & s \\ c & d \end{vmatrix} \begin{vmatrix} x \\ y \end{vmatrix} \quad (33)$$

where (x, y) and (x', y') are the original and transformed Cartesian coordinates and r, s, t, u are real numbers. The equivalent transform for complex numbers is given by:

$$X' = A * X + B * X^{\sim} \quad (34)$$

where X and X' are original and transformed complex numbers and A and B are complex numbers which determine the transform.

Let us start by proving an equivalence exists between equations (33) and (34). Let F_r be the real part of F and F_i the imaginary part of F for any complex number F . Equating the real and imaginary parts of equation

(34) we obtain:

$$\mathbf{X}_r' = (A_r + B_r) * X_r + (B_i - A_i) * X_i$$

$$\mathbf{X}_i' = (A_i + B_i) * X_r + (A_r - B_r) * X_i$$

Mapping $\mathbf{X}_r'$ to $\mathbf{x}'$, $\mathbf{X}_i'$ to $\mathbf{y}'$, X_r to x , and X_i to y , the following equations connecting r, s, t, u and A, B are obtained:

$$r = A_r + B_r$$

$$s = B_i - A_i$$

$$t = A_i + B_i$$

$$u = A_r - B_r$$

and conversely:

$$A_r = (r + u)/2$$

$$A_i = (t - s)/2$$

$$B_r = (r - u)/2$$

$$B_i = (s + t)/2$$

which proves the equivalence by explicit construction.

Next the values of A and B are found which map the complex number X_1 onto $\mathbf{X}_1'$ and the complex number X_2 onto $\mathbf{X}_2'$. Equation (34) gives:

$$\mathbf{X}_1' = A * X_1 + B * X_1^* \quad (35)$$

$$\mathbf{X}_2' = A * X_2 + B * X_2^* \quad (36)$$

These are regarded as two simultaneous equations with A and B as the unknowns. These are solved to obtain:

$$A = (\mathbf{X}_2' * X_1^* - X_2^* * \mathbf{X}_1') / (X_2 * X_1^* - X_1 * X_2^*)$$

$$B = (\mathbf{X}_1' * X_2 - X_2' * X_1) / (X_2 * X_1^* - X_1 * X_2^*)$$

EFFICIENCY CONSIDERATIONS

The estimation of efficiency is now considered, followed by one practical example taken from the geometric calculations above. Only an indication is provided, since timing details will depend on program logic, language, machine hardware and possibly the display. Assuming one is convinced of the advantages of hexagonal pixels and their integer addressing, two alternatives arise. The geometric calculations could be performed in one of the two ways described in the section on 'Implementation', or the pixel addresses could be converted into Cartesian form, the calculations performed with the Cartesian coordinates in the usual fashion, and then the resultant address or addresses converted back to oblique axes. Which is more efficient? The latter course always involves floating-point arithmetic since $3^{1/2}$ is a factor, while it may be possible to use integer arithmetic alone for the former. In this simplified analysis the advantage of our method is neglected, and no difference is assumed between integer and floating-point instruction times. Each addition and subtraction instruction is counted as being of equal time T_a . Multiplication instructions take

time T_m and division instructions time T_d . Usually

$$T_d > T_m > T_a.$$

The formula for converting the pixel address (m, n) to the Cartesian address (x, y) is:

$$y = n * 3^{1/2}/2$$

$$x = m + n/2$$

Assuming that $3^{1/2}/2$ is calculated once at the beginning of the program at negligible cost, and that our computer language does not permit us to regard division by 2 as a shift operation, the conversion involves one multiplication, one addition and one division for each pixel and is therefore of length:

$$C = T_d + T_m + T_a \quad (37)$$

The formula for reconversion of (X, Y) to (M, N) after the Cartesian arithmetic has been performed is:

$$N = Y * 2/3^{1/2}$$

$$M = X - N/2$$

This takes the same time C , assuming that $2/3^{1/2}$ is already known.

The times t_a, t_s, t_m, t_c, t_d are now considered for complex addition, subtraction, multiplication, conjugation and division derivable from equations (1), (2), (4), (5) and (7) respectively:

$$t_a = 2 * T_a$$

$$t_s = 2 * T_a$$

$$t_m = 4 * T_m + 3 * T_a$$

$$t_c = 2 * T_a$$

$$t_d = 2 * T_d + 6 * T_m + 4 * T_a \quad (38)$$

As an example, the times involved for the linear transform of the last section are calculated, using method (i) of the section on 'Implementation'. The formula for transforming one pixel using complex arithmetic is given by equation (34). This involves two complex multiplications, one conjugation and one complex addition. The time taken is therefore:

$$2 * t_m + t_c + t_a$$

Using equations (38) this can be converted to a time expressed in machine instructions:

$$T = 8 * T_m + 10 * T_a$$

The equivalent formula using Cartesian axes is given by equation (33). This involves four multiplications and two additions. We add the time taken in converting the hexagonal pixel address to Cartesian and back, $2 * C$ as given in equation (37), to obtain a total:

$$T' = 2 * T_d + 6 * T_m + 4 * T_a$$

In the approximate analysis used here there is little

difference between T and T' . Other geometric operations can be analysed in a similar way.

However, if it is desired to obtain the maximum efficiency for a calculation, perhaps because it will be used many times in the program, the efficiency can be improved by using method (ii) in the section on 'Implementation'. Method (ii) is now used for the linear transform. For A and B as in equation (34) we set:

$$A = (M_A, N_A)$$

$$B = (M_B, N_B)$$

If the pixel to be transformed has address (m, n) before transformation and (m', n') after transformation, we have:

$$(m', n') = (M_A, N_A) * (m, n) + (M_B, N_B) * (m + n, -n)$$

where equation (5) has been used to find the complex conjugate of (m, n) . Equation (4) is then used to perform the complex multiplications finding:

$$m' = (M_A + M_B) * m + (M_B + N_B - N_A) * n$$

$$n' = (N_A + N_B) * m + (M_A + N_A - M_B) * n$$

The expressions $(M_A + M_B)$, $(M_B + N_B - N_A)$, $(M_A + N_A - M_B)$ can all be calculated once at the beginning, after which the general linear transform will be as efficiently applied to the pixel addresses with oblique axes as it would be for square pixels using Cartesian axes, and also much faster than converting the hexagonal pixel addresses (m, n) to Cartesian addresses and using the Cartesian formula.

SUMMARY AND CONCLUSIONS

Hexagonal pixels are better than squares in most spatial applications because they are uniformly adjacent to their neighbours and thus are connected to them in only one way. They also have better angular resolution, the average distance of a point inside the pixel from the centroid is less and they have more axes of symmetry. However, the pixel address for squares, (x, y) , decomposes easily into the two parts required for the Cartesian approach to geometry, and the results of such a calculation, (x', y') , can be easily converted to a pixel address by truncation or rounding to integers.

This paper has shown that it is possible to address hexagonal pixels with two integers (m, n) if oblique axes at 60° to each other are used, and provide equally elegant algorithms for geometrical calculations. The pixel addresses correspond to complex numbers and complex-number operations on these produce a geometry for deriving the algorithms. These algorithms are called set B: for example the equation of a circle, equation (25). The primitive complex-number operations in them are complex addition, subtraction, multiplication, division, conjugation and the extraction of square roots. A second set of algorithms, A, for example the equation for complex multiplication, equation (4), defines these primitives in terms of the

pixel addresses (m, n) . The pixel addresses are decomposed into the two integers m and n , and then the conventional arithmetic instructions of the conventional computer, such as add, subtract, multiply and divide for integers or floating point numbers are used. The programmer can choose between implementing set A as subroutines, or, at the expense of greater programming effort, substituting the equations in symbolic form into set B, and simplifying as far as possible to provide faster execution. In either case the combined set of algorithms, A and B, takes values of the hexagonal pixel address (m, n) as arguments, and produces new values (m', n') using the conventional arithmetic instructions of the conventional computer. The new values (m', n') can be easily converted to pixel addresses by truncation or rounding to integers.

This paper has shown by numerous examples that it is as easy to provide geometrical algorithms using the geometrical properties of complex numbers as it is using the conventional breakdown of the Cartesian (x, y) addressing into x and y . The new expressions which result are no more complicated, and in many cases simpler and more elegantly symmetric, as can be seen by reworking the geometrical algorithms of set B using Cartesian coordinates¹⁷. It is conjectured that other geometrical problems will also yield easily and elegantly to the new approach. Once the geometrical algorithm has been obtained in terms of complex numbers it is simple to convert it into ordinary arithmetic operations on pixel addresses. In particular, complex addition, subtraction and multiplication on integer hexagonal pixel addresses always gives exact integer answers, as in the case of square pixel addresses and ordinary addition, subtraction and multiplication. In short, digital geometry is as easily, accurately and efficiently done using hexagons.

ACKNOWLEDGEMENTS

This work was partly supported under NERC Contact No. F60/G6/12.

REFERENCES

- 1 Golay, M J E 'Hexagonal parallel pattern transformations' *IEEE Trans. Comput.* Vol C-18 No 3 (August 1969) pp 733-740
- 2 Preston, K 'Features extraction by Golay hexagonal pattern transformations' *IEEE Trans. Comput.* Vol C-20 No 9 (September 1971) pp 1007-1014
- 3 Bell, S B M, Diaz, B M and Holroyd, F C 'The HoR quadtree: an optimal structure based on a non-square 4-shape' *Proc. Institute of Mathematics and Its Applications Conference on Remote Sensing, UK, May 1986* Oxford University Press, Oxford, UK (1988)
- 4 Rosenfeld, A 'Digital topology' *Am. Math. Monthly* Vol 86 No 8 (1979) pp 621-630
- 5 Rosenfeld, A 'Connectivity in digital pictures' *J. ACM* Vol 17 No 1 (January 1970) pp 146-160
- 6 Deutsch, E S 'On parallel operations on hexagonal arrays' *IEEE Trans. Comput.* (October 1970) pp 982-983

- 7 **Mendelson, B** *Introduction to topology* Third edition. Allyn and Bacon, Inc., Boston, MA, USA (1975)
- 8 **Massey, S W** *Singular homology theory* Graduate texts in mathematics, Springer-Verlag New York Inc., NY, USA (1980)
- 9 **Deutsch, E S** 'Thinning algorithms on rectangular, hexagonal, and triangular arrays' *Commun. ACM* Vol 15 No 19 (September 1972) pp 827–837
- 10 **Luczak, E and Rosenfeld, A** 'Distance on a hexagonal grid' *IEEE Trans. Comput.* (May 1976) pp 523–533
- 11 **Mersereau, R M** 'The processing of hexagonally sampled two-dimensional signals' *Proc. IEEE* Vol 67 No 6 (1979) pp 930–949
- 12 **Cox, J A** 'Point-source location using hexagonal detector arrays' *Opt. Eng.* Vol 26 No 1 (1987) pp 69–74
- 13 **Gibson, L and Lucas, D** 'Vectorisation of raster images using hierarchical methods' *Comput. Graphics and Image Process.* Vol 20 (1982) pp 82–89
- 14 **Bell, S B M, Diaz, B M, Holroyd, F C and Jackson, M J** 'Spatially referenced methods of processing raster and vector data' *Image Vision Comput.* Vol 1 No 4 (1983) pp 211–220
- 15 **Samet, H** 'The quadtree and related hierarchical data structures' *ACM Comput. Surveys* Vol 16 (1984) pp 187–260
- 16 **Bell, S B M, Diaz, B M and Holroyd, F C** 'Lattice rings in two dimensions' **Bell, S B M (Ed.)** *Tesseral newsletter* No 3 (March 1987)
- 17 **Bowyer, A and Woodwark, J** 'A programmer's geometry' Butterworths, London, UK (1983)